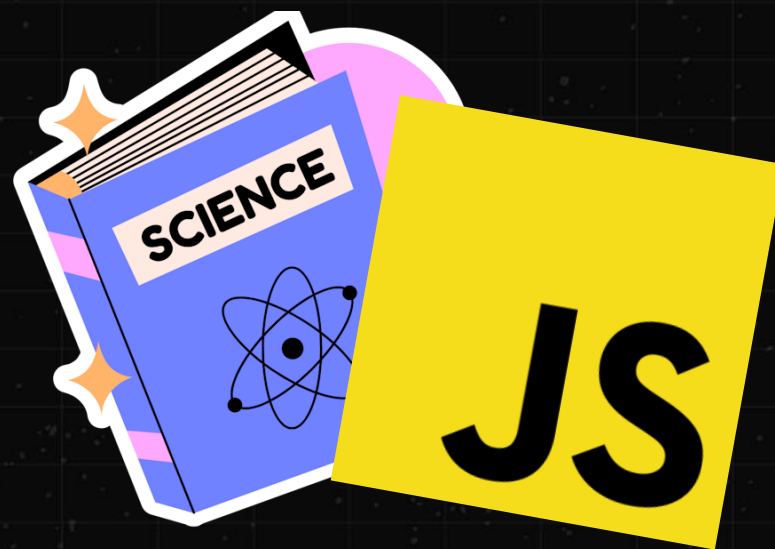


ЛЕКЦИЯ #8

CS BO FRONTEND



ЧТО СЕГОДНЯ НА ПОВЕСТКЕ

- * Будем говорить про различные **форматы данных**
- * **Текстовое и бинарное** представление
- * Маршалинг/демаршалинг и сериализацию/парсинг
- * Познакомимся с конкретными примерами

В ЧЕМ ОСОБЕННОСТЬ ТЕКСТОВОГО ПРЕДСТАВЛЕНИЯ?



ОНО НАСТОЛЬКО НАМ ПРИВЫЧНО

- * Что мы даже не задумываемся о том, **как это работает**
- * Мы работаем с текстом в мессенджерах и на веб-сайтах
- * Работая в Word/Excel или любым другим похожим ПО
- * Исходный код на любом ЯП – это просто текстовые файлы
- * Даже при загрузке компьютера мы видим текст
- * В действительности, никакого текста в компьютере нет – **только поток байт**

НО Я ЖЕ ВИЖУ ТЕКСТ,
А НЕ ПРОСТО БАЙТЫ
ПРИ ЧТЕНИИ .TXT
ФАЙЛА?



ВЕРНО, НО ТУТ ЕСТЬ НЮАНС

- * Любые данные в компьютере представляются потоком байт и текст не исключение
- * Любой «текст» должен быть закодирован в двоичный вид тем или иным способом
- * Текстовый редактор декодирует данные и отображает их на экране

КАК ПРОИСХОДИТ ОТОБРАЖЕНИЕ ТЕКСТА?



КАК БЫЛО РАНЬШЕ

- * Первые компьютеры имели **аппаратную поддержку различных кодовых таблиц** в своём видеоадаптере (CGA, EGA, VGA и так далее)
- * Грубо говоря, задача видеоадаптера была **нарисовать символ на экране монитора** согласно его коду и используемой кодовой таблице
- * Программа-шрифты тоже **придумали не сразу**

КАК СЕЙЧАС

- * Кодовые таблицы заменились Юникодом и универсальными кодировками (например, UTF-8 или UTF-16)
- * Современные UEFI поддерживают юникод на аппаратном уровне
- * За визуальное отображение символа отвечает программа-шрифт и подсистема ОС или отдельная библиотека
- * Видеокарта рендерит пиксели, а не символы

ШРИФТЫ

- * Представляют собой данные/форматы описывающие **как визуально отобразить множество конкретных символов**
- * Шрифт создается **под конкретную кодовую таблицу** (сейчас это Юникод)
- * И может содержать **глифы не для всех символов**
- * А может вообще **отображать другие изображения** (например, «Segoe UI Emoji» содержит эмодзи 😊 , 🚀)

А ЕСЛИ ШРИФТ
НЕ ПОДДЕРЖИВАЕТ
КОНКРЕТНЫЙ
СИМВОЛ?



ЕСТЬ НЕСКОЛЬКО ВАРИАНТОВ

- ✱ Можно попытаться отобразить символ **используя другой «запасной» шрифт**
- ✱ Или **просто отобразить спецсимвол**, например, □
- ✱ Эта логика **может меняться от конкретной программы**

ЗАДАЧА ТЕКСТОВОГО РЕДАКТОРА

- * Понять каким образом закодирован текст и декодировать его (во времена «до Юникода» это было большой проблемой)
- * **«Отрендерить» текст используя выбранные шрифты** с корректным отображением спецсимволов (переводы строк и так далее)
- * Программа редактор может использовать свой универсальный формат кодирования, но **при сохранении она должна закодировать файл согласно требованиям**

ВЕРНЕМСЯ К ТЕКСТОВОМУ ФОРМАТУ

- * Например, число 255 можно закодировать и сохранить/передать как текст
- * В кодировке UTF-8 такое число займет 3 байта, а в UTF-16 уже 6 байт
- * При этом, если потом мы захотим работать с этим числом, как с «числом», то нам **придется написать программу-преобразователь в формат числа**
- * Это же число **можно закодировать в прямой код** используя всего один байт

ПОЧЕМУ ТАКОЙ ОВЕРХЕД?



ИЗ-ЗА КОСВЕННОСТИ

- ✱ Мы кодируем наши данные в виде текста, а потом кодируем текст в байты
- ✱ Благодаря такому подходу у нас появляется возможность легко **прочитать/отредактировать эти данные** без специального ПО
- ✱ Но за это мы платим памятью и необходимостью декодировать данные из формата строки в нужное нам представление при чтении

**ЭФФЕКТИВНЕЕ
ПРЕДСТАВЛЯТЬ
ДАННЫЕ СРАЗУ
В БИНАРНОМ ВИДЕ?**



В ТОЧКУ

- * Бинарное представление позволяет выбрать такую схему хранения данных, которая позволит **минимизировать объем памяти и работать быстрее**
- * Поэтому, например, стандарт HTTP2 отказался от текстового формата
- * Разумеется, **бинарное представление само по себе не панацея** – важно как именно вы представляете данные в памяти
- * Но, в любом случае, **вы избавляетесь от лишней косвенности** из-за текста

ДАВАЙТЕ СРАВНИМ

- * Возьмем небольшой JSON-файл и его аналог в бинарном виде в формате [MessagePack](#)
- * Разумеется, если взять другие форматы, то результат будет отличаться

Текст в UTF-8, 366 байт

```
{
  "total": 3,
  "data": [
    {
      "id": 1,
      "name": "Анна",
      "email": "anna@example.com",
      "active": true
    },
    {
      "id": 2,
      "name": "Петр",
      "email": "petr@example.com",
      "active": false
    },
    {
      "id": 3,
      "name": "Мария",
      "email": "maria@example.com",
      "active": true
    }
  ]
}
```

Бинарный формат, 167 байт

```
82 a5 74 6f 74 61 6c 03 a4 64 61 74 61 93 84 a2 69 64 01 a4 6e 61 6d 65 a8 d0 90 d0
bd d0 bd d0 b0 a5 65 6d 61 69 6c b0 61 6e 6e 61 40 65 78 61 6d 70 6c 65 2e 63 6f
6d a6 61 63 74 69 76 65 c3 84 a2 69 64 02 a4 6e 61 6d 65 a8 d0 9f d0 b5 d1 82 d1
80 a5 65 6d 61 69 6c b0 70 65 74 72 40 65 78 61 6d 70 6c 65 2e 63 6f 6d a6 61 63
74 69 76 65 c2 84 a2 69 64 03 a4 6e 61 6d 65 aa d0 9c d0 b0 d1 80 d0 b8 d1 8f a5 65
6d 61 69 6c b1 6d 61 72 69 61 40 65 78 61 6d 70 6c 65 2e 63 6f 6d a6 61 63 74 69
76 65 c3
```

Текст в UTF-8 в бинарном виде, 366 байт



```
7b 0a 20 20 20 20 22 74 6f 74 61 6c 22 3a 20 33 2c 0a 20 20 20 20 22 64 61 74 61
22 3a 20 5b 0a 20 20 20 20 20 20 20 20 20 7b 0a 20 20 20 20 20 20 20 20 20 20
22 69 64 22 3a 20 31 2c 0a 20 20 20 20 20 20 20 20 20 20 22 6e 61 6d 65 22
3a 20 22 d0 90 d0 bd d0 bd d0 b0 22 2c 0a 20 20 20 20 20 20 20 20 20 20 22
65 6d 61 69 6c 22 3a 20 22 61 6e 6e 61 40 65 78 61 6d 70 6c 65 2e 63 6f 6d 22 2c
0a 20 20 20 20 20 20 20 20 20 20 20 20 22 61 63 74 69 76 65 22 3a 20 74 72 75 65
0a 20 20 20 20 20 20 20 20 7d 2c 0a 20 20 20 20 20 20 20 20 7b 0a 20 20 20 20 20
20 20 20 20 20 20 22 69 64 22 3a 20 32 2c 0a 20 20 20 20 20 20 20 20 20 20 20
20 22 6e 61 6d 65 22 3a 20 22 d0 9f d0 b5 d1 82 d1 80 22 2c 0a 20 20 20 20 20 20
20 20 20 20 20 22 65 6d 61 69 6c 22 3a 20 22 70 65 74 72 40 65 78 61 6d 70 6c
65 2e 63 6f 6d 22 2c 0a 20 20 20 20 20 20 20 20 20 20 20 22 61 63 74 69 76 65
22 3a 20 66 61 6c 73 65 0a 20 20 20 20 20 20 20 20 20 7d 2c 0a 20 20 20 20 20 20
20 7b 0a 20 20 20 20 20 20 20 20 20 20 20 20 22 69 64 22 3a 20 33 2c 0a 20 20 20
20 20 20 20 20 20 20 20 22 6e 61 6d 65 22 3a 20 22 d0 9c d0 b0 d1 80 d0 b8 d1
8f 22 2c 0a 20 20 20 20 20 20 20 20 20 20 20 20 22 65 6d 61 69 6c 22 3a 20 22 6d
61 72 69 61 40 65 78 61 6d 70 6c 65 2e 63 6f 6d 22 2c 0a 20 20 20 20 20 20 20 20
20 20 20 20 22 61 63 74 69 76 65 22 3a 20 74 72 75 65 0a 20 20 20 20 20 20 20 20
7d 0a 20 20 20 20 5d 0a 7d 0a
```

Бинарный формат, 167 байт



```
82 a5 74 6f 74 61 6c 03 a4 64 61 74 61 93 84 a2 69 64 01
a4 6e 61 6d 65 a8 d0 90 d0 bd d0 bd d0 b0 a5 65 6d 61 69
6c b0 61 6e 6e 61 40 65 78 61 6d 70 6c 65 2e 63 6f 6d a6
61 63 74 69 76 65 c3 84 a2 69 64 02 a4 6e 61 6d 65 a8 d0
9f d0 b5 d1 82 d1 80 a5 65 6d 61 69 6c b0 70 65 74 72 40
65 78 61 6d 70 6c 65 2e 63 6f 6d a6 61 63 74 69 76 65 c2
84 a2 69 64 03 a4 6e 61 6d 65 aa d0 9c d0 b0 d1 80 d0 b8
d1 8f a5 65 6d 61 69 6c b1 6d 61 72 69 61 40 65 78 61 6d
70 6c 65 2e 63 6f 6d a6 61 63 74 69 76 65 c3
```

РЕФЛЕКСИРУЕМ

- * В данном примере можно было более «эффективный» текстовый формат, так и бинарный – **но суть бы это не поменяло**
- * Текст – **это тоже бинарные данные** и когда мы кодируем данные в текст, то мы добавляем косвенность (двойное кодирование)
- * В общем случае прямое **бинарное представление всегда будет лучше** с точки зрения эффективности (память и скорость)
- * А с текстом **удобнее работать и отлаживаться** (эффективность разработки)

СЖАТИЕ СПЕШИТ НА ПОМОЩЬ

- * Улучшить ситуацию с объемом текстовой информации могут различные алгоритмы и форматы хранения сжатых данных
- * Но надо понимать, что за это мы платим скоростью доступа к данным: явные фазы компрессии и декомпрессии
- * Степень сжатия – чем выше, тем больше нагружает CPU при сжатии
- * Бинарные файлы тоже можно сжимать таким образом

ТАК ЧТО
ВЫБРАТЬ?



ЗАВИСИТ ОТ ВИДА ДАННЫХ

- * Если ваша информация «вписывается» в текст или является текстом – строки или отдельные символы
- * Если вам важна возможность простого чтения и редактирования, например, файлы конфигурации
- * А вот звук или изображение требуют особого способа «рендеринга» – поэтому нет смысла тут использовать текст
- * С другой стороны, векторную графику удобнее представлять в текстовом формате (например, SVG)

ЗАВИСИТ ОТ ОБЪЕМА ДАННЫХ

- * Если данные **занимают весомый процент на диске или долго передаются по сети**, то стоит задуматься об оптимизации способа хранения данных
- * Но **всегда учитывайте сжатие** (и не забывайте про него)
- * Не всегда стоит сразу бежать в сторону бинарных форматов – иногда эффективнее будет **просто взять другой текстовый формат**
- * Например, XML → JSON, JSON → CSV

В ОБРАТНУЮ СТОРОНУ ТОЖЕ МОЖНО

- * Любой бинарный файл можно закодировать в виде текста:
закодировать как поток символов-чисел в HEX или использовать стандарт [Base64](#)
- * Учитывайте, что в таком случае увеличивается объем хранимой информации
(для Base64 в среднем на 35%)
- * Скорость кодирования/декодирования также страдает из-за косвенности:
бинарные данные → текст → бинарные данные

НО ЗАЧЕМ
НАМ ЭТО
ДЕЛАТЬ?



ЕСТЬ СЦЕНАРИИ

- * Вы ограничены использованием только текста
- * Например, URL ресурса должен быть текстовым
- * Вы встраиваете бинарный файл внутрь текстового файла
- * Например, встраиваете небольшие изображения сразу в CSS файл, чтобы не делать отдельные запросы

URI И URL

- * Согласно стандартам Интернета у **каждого ресурса есть URI и URL**
- * URI (Uniform Resource Identifier) – **имя или адрес** ресурса
- * URL (Uniform Resource Locator) – **адрес, по которому ресурс можно получить**
- * **Все URL являются URI** (каждый адрес – это идентификатор)
- * **Не все URI являются URL** (идентификатор может быть просто именем, без указания, где его взять)

НАПРИМЕР

- * <https://example.com/page.html> – URL и URI
- * <mailto:john@example.com> – URL и URI
- * <urn:isbn:0451450523> – только URI

DATA:URI

- * URI ресурса в котором **находятся сами данные** (в том числе бинарные)
- * Для бинарных данных нужно использовать Base64 (URI – всегда текст)

```
data:image/png;base64,iVBORw0KGgoAAAANSUhEUgAAACgAAA  
AoCAYAAACM/rhtAAAWUIEQVRyR+3SwQkAIBAEsbP/otUe5iMS8  
Tsgcdeeuffdszwwfg7BCDgECVaB2tsgwSpQexskWAVqb4MEq0Dtb  
ZBgFai9DRKsArW3QYJVoPY2SLAK1N4Gvxc88nhP2TSkVqEAAAAS  
UVORK5CYII=
```

second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

СТОЯНА, СТОЯНААА!

ПОЙМИТЕ СУТЬ

- * Любые данные в компьютере являются бинарными (хранятся как поток байт)
- * Текстовое представление важно людям, так как нам **сложно читать сырые байты**
- * Когда мы кодируем данные в виде текста – мы создаем косвенность:
данные закодированы в текст, а текст в бинарное представление
- * Главное преимущество – **простота чтения и редактирования**
- * Прямое бинарное представление в ряде случаев **может быть сильно эффективнее** ([например](#))

ПОГОВОРИМ ПРО ФОРМАТЫ ДАННЫХ

- * Мы знаем, что программы по-разному представляют свои типы данных в памяти
- * С одной стороны, это связано с тем, на каком ЯП написана программа
- * А с другой, с тем, как это придумали сами создатели этой программы
- * Такое представление оптимизировано для работы CPU и памяти
- * Но у нас все равно есть потребность преобразовать данные в формат для хранения данных на диске или передаче по сети или в другой процесс
- * И здесь открывает множество сценариев и решений

МАРШАЛИНГ И ДЕМАРШАЛИНГ ДАННЫХ

- * Маршалингом называется **процесс преобразования данных из памяти программы** в формат, пригодный для сохранения на диске или передаче по сети
- * Демаршалинг – обратный **процесс с восстановлением памяти программы** через чтение такого файла
- * Ключевая особенность в том, что мы должны полностью сохранить все состояние данных и **иметь возможность восстановить его точь-в-точь**

ФОРМАТЫ МАРШАЛИНГА ПРИВЯЗАНЫ К ЯП

- * У многих ЯП такая фича является частью стандартной библиотеки
- * Например, [java.io.Serializable](#) в Java
- * Плюсом такого подхода является **простота и удобство**
- * А минусом **переносимость и проблемы с безопасностью**
- * Поэтому на практике часто используют **сериализацию в платформонезависимый формат**

СЕРИАЛИЗАЦИЯ И ДЕСЕРИАЛИЗАЦИЯ

- * Логически тут все очень **похоже** на **маршалинг** и **демаршалинг**
- * Но с той разницей, что мы **сохраняем не все состояние, а только данные**
- * Выбранный формат для сериализации может просто **не поддерживать все возможные типы данных нашего ЯП**
- * Например, в JSON нет отдельного формата для дат – даты нужно кодировать либо как строки, либо как числа

НАПРИМЕР



```
const data = JSON.stringify({date: new Date()});  
  
const parsedData = JSON.parse(data);  
  
console.log(typeof parsedData.date === "string");
```

ПАРСИНГ

- * Процесс десериализации **часто называют парсингом**
- * Хотя парсинг – это по сути **процесс анализа содержимого** в некотором формате для последующей обработки
- * Когда мы загружаем исходный код в компилятор или VM, то **он приходит как обычный текст, а затем уже разбирается на части**
- * Происходит процесс проверки ошибок и построения структур для дальнейшей работы

ТАК ЧТО ТАМ
С ФОРМАТАМИ?



JSON

- * Главная рабочая лошадка в Web и любимый формат фронтендеров
- * Родился на основе **литеральной формы объектов ES3**, но быстро стал популярным и сейчас считается форматом не привязанным к языку
- * Текстовый формат с **поддержкой нескольких видов информации**: словари {"**ключ**": **значение**}, массивы [], числа, строки "", **true**, **false**, **null**
- * API для сериализации и парсинга **входит в стандартную библиотеку JS**

НАПРИМЕР

```
{
  "total": 1,
  "data": [
    {
      "id": 1,
      "name": "Анна",
      "active": true,
      "skills": null
    }
  ]
}
```

ПЛЮСЫ JSON

- * Формат в меру удобен для чтения человеком и прост для парсинга
- * Подходит как для файлов конфигурации, так и для обмена данными
- * Достаточно экономичный дизайн по сравнению с XML
- * Несколько встроенных типов информации
- * Очень эффективная поддержка в JS из коробки

МИНУСЫ JSON

- * Один числовой формат для целых и дробных чисел, проблема с большими целыми
- * Нет отдельного типа для хранения даты и времени
- * Не поддерживаются комментарии
- * Плохая поддержка многострочных литералов
- * Любая бинарная информация должна кодироваться в текст
- * Из коробки не поддерживается потоковая обработка

СПОРНЫЕ МОМЕНТЫ

- * JSON не имеет стандартной поддержки схем данных
- * С одной стороны это хорошо для гибкости при прототипировании, но из-за этого файлы имеют больший объем и есть проблемы со стабильностью
- * Если JSON успешно прошел парсинг – это не значит, что данные в нем валидны с точки зрения логики приложения
- * Например, что есть все нужные поля и их типы корректны – это должно проверяться отдельной логикой
- * Про совместимость данных тоже должен думать сам разработчик

ПОЧЕМУ
ОТСУТСТВИЕ
СХЕМЫ ДЕЛАЕТ
ФАЙЛ БОЛЬШЕ?



КОДИРОВАНИЕ КЛЮЧЕЙ

- * Проблема в том, что из-за отсутствия схемы приходится кодировать ключи объектов в самих данных
- * Например, { "id": 1, "name": "Анна", "active": true, "skills": null }
- * Если объектов много, то избыточность растет как снежный ком
- * Сжатие уменьшает эту проблему, но многие забывают сжимать JSON

ПОДДЕРЖКА В JS

- * Типы `number` | `boolean` | `null` поддерживают сериализацию из коробки
- * `Date` сериализуется в строку в формате **ISO 8601** (при парсинге остается строкой)
- * Массивы и объекты тоже поддерживают её, но при условии, что внутри **все значения** сериализуемы в JSON
- * Добавить поддержку сериализуемости можно через определение `toJSON`
- * Либо задав `replacer` в `JSON.stringify`

НАПРИМЕР

```
const data = {  
  name: "Bob",  
  toJSON: () => 42  
};  
  
console.log(JSON.stringify(data) === "42");
```

```
const BIGINT_PREFIX = "ⓔbigint:";

const data = JSON.stringify({value: 42n}, encodeBigint);

// '{"value": "ⓔbigint:42"}'
console.log(data);

// {value: 42n}
console.log(JSON.parse(data, decodeBigint));

function encodeBigint(key, value) {
  if (typeof value === "bigint") {
    return BIGINT_PREFIX + String(value);
  }

  return value;
}

function decodeBigint(key, value) {
  if (typeof value === "string" && value.startsWith(BIGINT_PREFIX)) {
    return BigInt(value.replace(BIGINT_PREFIX, ""));
  }

  return value;
}
```

РЕФЛЕКСИРУЕМ

- * С помощью определения метода `toJSON` или передачи доп. аргументов в API `JSON` можно **расширить поддержку новых типов данных**
- * Проблема в том, что **это не универсальный подход** – вам придется реализовывать эту логику **как на клиенте, так и на сервере**
- * Не факт, что используемая на сервере библиотека для работы с JSON (если сервер не на JS) поддерживает такие возможности
- * У `JSON.stringify` есть еще 3-й аргумент, позволяющий **делать красивое форматирование с отступами**

ПОЧТИ JSON

- * У формата JSON есть множество подмножеств, например, [JSON5](#)
- * JSON5 делает JSON **более удобным для конфигов**:
появились комментарии, не обязательно ставить двойные кавычки и так далее
- * **Не используйте его для передачи данных** –
скорость парсинг и сериализации будет намного ниже

СХЕМЫ В JSON

- * Нatively JSON не поддерживает схемы, поэтому **мы не можем изящно решить проблему избыточности**
- * Но схема это еще отличная документация формата и **возможность валидировать данные**
- * Есть сторонние проекты, которые добавляют схемы для JSON
- * Например, [JSON Schema](#)

А КАКИЕ КОНКУРЕНТЫ У JSON?



СРЕДИ ТЕКСТОВЫХ ФОРМАТОВ

- * Для создания файлов конфигурации куда удобнее [TOML](#) или [YAML](#)
- * Для передачи сложной информации (векторная графика, разметка и так далее) куда удобнее использовать [XML](#)
- * Также нельзя списывать со счетов простой [CSV](#) – он очень эффективен для больших наборов простой информации и поддерживает поточную обработку

XML

- * Расширяемый язык разметки – **метаязык для создания языков разметки**
- * **Многие языки разметки** явно или косвенно сделаны на основе XML: HTML, SVG (векторная графика), MathML (математические формулы)
- * Вывод – XML не самостоятельный язык разметки данных, а **набор синтаксических правил для создания таких языков**


```
<?xml version="1.0" encoding="utf-8"?>
<!DOCTYPE recipe>
<recipe name="хлеб" preptime="5min" cooktime="180min">
  <title>
    Простой хлеб
  </title>
  <composition>
    <ingredient amount="3" unit="стакан">Мука</ingredient>
    <ingredient amount="0.25" unit="грамм">Дрожжи</ingredient>
    <ingredient amount="1.5" unit="стакан">Тёплая вода</ingredient>
  </composition>
  <instructions>
    <step>
      Смешать все ингредиенты и тщательно замесить.
    </step>
    <step>
      Закрыть тканью и оставить на один час в тёплом помещении.
    </step>
    <!--
    <step>
      Почитать вчерашнюю газету.
    </step>
    - это сомнительный шаг...
    -->
    <step>
      Замесить ещё раз, положить на противень и поставить в духовку.
    </step>
  </instructions>
</recipe>
```

СИЛЬНЫЕ СТОРОНЫ XML

- * Очень продуманный стандарт и богатый синтаксис
- * Огромная экосистема смежных проектов – схемы, шаблонизаторы, языки запросов
- * Например, [XML Schema](#), [XSLT](#), [XPath](#), [XQuery](#)
- * Выгрузку в XML умеет любое «уважающее себя» ПО
- * В старых ПО JSON может не поддерживаться, но XML скорее всего будет

XML VS JSON

- * XML куда более богатый с точки зрения синтаксиса, но с другой стороны **в нем нет поддержки типов данных** (числа, логические и другие)
- * Вся информация хранится **в узлах и их атрибутах** как простой текст
- * XML куда **многословнее** и файлы в нем **весят больше** (в среднем в 2-4 раза)
- * **Парсинг XML сложнее и делается менее эффективно**
- * После парсинга JSON отображается на объекты/структуры ЯП, а **работа с XML обычно делается через специальный API**

С ДРУГОЙ СТОРОНЫ

- * Пока структура информации простая, то работать с ней в JSON человеку может быть и проще чем с XML
- * Но с ростом сложности **JSON очень быстро превращается в нечитаемое месиво**, а с XML работать по прежнему удобно
- * Для передачи «обычных» данных с сервера JSON куда лучше, но **любая разметка, векторная графика удобнее в XML**
- * Если вы делаете свой язык разметки для рендеринга на Canvas – используйте XML (верстать в JSON – мрак)

XML И JS

- * Поддержка в браузере появилась намного раньше JSON
- * Мы можем распарсить строку как XML и работать с ней через DOM API
- * Делать сложные запросы к данным через селекторы
- * А вот в Node.js поддержка добавляется с помощью сторонних библиотек

НАПРИМЕР

```
fetch('data.xml')  
  .then(response => response.text()) // Получаем XML как текст  
  .then(xmlString => {  
    const parser = new DOMParser();  
    const xmlDoc = parser.parseFromString(xmlString, "text/xml");  
    // Работаем с xmlDoc  
  });
```

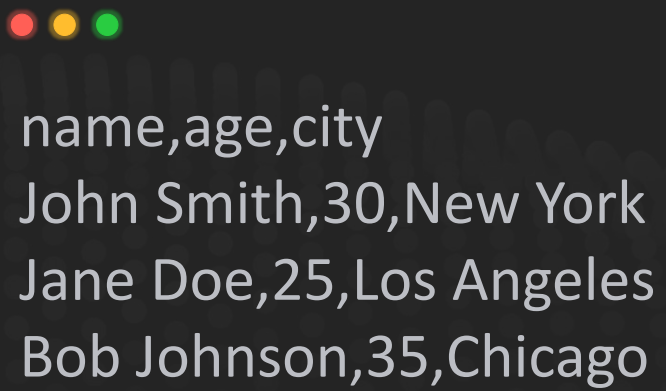
ВЫВОДЫ

- * XML многословнее и **файлы на нем весят больше**, чем на JSON, а парсинг – медленней
- * Отсутствие встроенных типов и маппинга на объекты ЯП – делает его **плохим кандидатом для формата передачи «обычных» данных**
- * Но если ваши данные должны идти с разметкой, то XML становится очень удобен в разработке и чтении
- * Так что не списывайте его со счетов

CSV

- * Еще один ультра популярный **текстовый формат данных**
- * Это **просто текст разбитый на строки через перевод строки и столбцы через запятую или другой символ**
- * В нем нет **никаких типов данных или схем** – все текст
- * **Очень простой и быстрый парсинг**

НАПРИМЕР



```
name,age,city  
John Smith,30,New York  
Jane Doe,25,Los Angeles  
Bob Johnson,35,Chicago
```


И В ЧЕМ ТУТ
ПРОФИТ?



НЕ СПЕШИТЕ С ВЫВОДАМИ

- * CSV – это **самое экономичное текстовое представление из возможных**
- * Если в JSON идут ключи, скобки и кавычки, то в CSV всего два разделителя
- * В среднем CSV в **3-4 раза меньше JSON**, а парсинг быстрее
- * Самое главное – нам **не нужно загружать CSV данные целиком**,
а просто обрабатывать построчно (например, логи могут весить десятки гигабайт)
- * Как и **не нужно загружать все данные для сериализации**
- * JSON тоже можно сделать поточным,
но это кастомное решение с кучей нюансов

CSV ПОДДЕРЖИВАЮТ ВСЕ

- * Стандартные утилиты командной строки
- * Excel и Google таблицы
- * Базы данных
- * Список больше чем у XML

МИНУСЫ ТОЖЕ ИМЕЮТСЯ

- * Очень простая структура: строки и столбцы и никакой вложенности
- * Можно использовать JSON внутри CSV столбцов, чтобы это преодолеть
- * Но читать такое станет еще сложнее
- * Проблемы с экранированием разделителя – если разделитель встречается в данных, то это надо специально обрабатывать
- * Сам разделитель не стандартизирован, никаких схем и типов

second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

СТОЯНА, СТОЯНААА!

ВЫДЫХАЕМ

- * Хороший архитектор должен знать много форматов данных и сценарии для их использования, а не использовать всегда один
- * Например, **писать логи в JSON очень плохая идея**: просто так в файл не записать новую строку, а парсинг гигабайтного лога будет той еще задачей
- * Или **делать язык разметки на основе JSON**

А ЧТО С **БИНАРНЫМИ** ФОРМАТАМИ?



ТУТ ВЫБОР КУДА ОБШИРНЕЕ

- * Но можно выделить **семейство форматов без схемы по принципу «бинарный JSON»**
- * **Основной профит в компактности данных** и возможности хранить двоичные данные без преобразования в строку
- * И **форматы со схемами** – которые позволяют еще сильнее скомпоновать данные, поддержать больше типов данных и возможности валидации
- * Мы познакомимся с несколькими из них

MESSAGEPACK

- * Популярная бинарная альтернатива JSON с реализацией на большом количестве ЯП
- * В дополнение к типам JSON появляются байты (любые бинарные данные), **целые числа разного размера и дробные с настраиваемой точностью**
- * Работает без схемы и в среднем файлы весят меньше на 15-20% (с учетом сжатия)
- * На отдельных данных **разница может быть куда больше**
- * Скорость парсинга и сериализации **зависит от библиотеки**
- * Для JS есть очень быстрая реализация с поддержкой поточной обработки



```
import { pack, unpack } from "msgpackr";

// Данные для сериализации
const data = {
  name: "Alice",
  age: 30,
  city: "New York",
  tags: ["developer", "blogger"],
  isActive: true
};

// Сериализация (объект → бинарный буфер)
const binaryData = pack(data);
console.log("Размер:", binaryData.byteLength, "байт");

// Десериализация (бинарный буфер → объект)
const restored = unpack(binaryData);
console.log("Восстановленный объект:", restored);
```


MESSAGEPACK VS JSON

- * Как бинарный формат, MessagePack совершенно не годится для файлов конфигурации
- * А вот для передачи данных он почти во всем лучше JSON
- * Проигрывает в удобстве отладки и поддержки разными инструментами
- * Очень легко перейти с JSON и обратно (поддержку JSON стоит оставить для отладки)
- * Оба формата не умеют в поточную обработку из коробки (только на уровне библиотек)
- * В JS имеется [очень хорошая реализация](#)

PROTOBUF

- * Бинарный формат данных от Google
- * Формат данных **описывается схемой на специальном IDL** (proto-файл)
- * Поддерживает богатый набор встроенных типов, а также **механизмы для прямой и обратной совместимостей**
- * За счет схемы **очень компактно упаковывает данные** (в среднем в 1.3-1.5 раза лучше MessagePack)
- * Скорость парсинга и сериализации **зависит от реализации**

person.proto

```
edition = "2024";

message Person {
  string name = 1;
  int32 id = 2;
  string email = 3;
}
```

```
npm i protobufjs
npm i protobufjs-cli --save-dev
npx pbjs -t static-module -w commonjs -o person.js person.proto
```

```
const { Person } = require("./person");

const data = { name: "Alice", id: 42, email: "alice@example.com" };

const encoded = Person.encode(data).finish();
const decoded = Person.decode(encoded);

console.log(encoded.byteLength);
console.log(decoded);
```

РЕФЛЕКСИРУЕМ

- * Мы описали **формат наших данных** в специальном proto-файле
- * Затем сгенерировали по этому файлу JS библиотеку, которая **содержит в себе логику сериализации и десериализации данных**
- * Подключаем её как библиотеку и используем
- * Такой же подход используется и с другими ЯП

ЗАМЕЧАНИЕ

- * Библиотека [protobufjs](#) поддерживает динамическую загрузку без предварительной компиляции proto-файлов, но **это не для prod**
- * Библиотека `protobufjs-cli` поддерживает генерацию как для браузера, так и для Node.js или сборщика
- * Генерацию с использованием ES6+ кода и генерацию **.d.ts файлов**
- * **Это сторонняя библиотека**

А ЧТО ЗА
ЦИФРЫ В
PROTO-ФАЙЛЕ?



ЭТО ТЕГИ ПОЛЕЙ

- * Использование схемы позволяет **не кодировать ключи в самих данных**
- * С другой стороны, **у нас есть проблема обратной и прямой совместимости** при обновлении proto-схемы
- * Обратная совместимость – новая схема должна работать **для файлов закодированной по старой схеме**
- * Прямая совместимость – старая схема должна работать **для файлов закодированной по новой схеме**

second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

СТОЯНА, СТОЯНААА!

БЕЗ ПАНИКИ, СЕЙЧАС РАЗБЕРЕМСЯ

- * В форматах без схемы **любое** изменение формата данных может привести к последствиям
- * Если **клиент** рассчитывает, что значение по ключу есть, а его нет, то все сломается
- * Мы должны **самостоятельно** в коде обрабатывать такие ситуации
- * Наличие схемы данных **позволяет** сделать комплексное решение

КАК ЭТО СДЕЛАНО В PROTOBUF

- * Можно добавлять новые поля в схему или менять их местами – **главное, не менять их теги, а для новых – использовать новые теги**
- * Старый код будет **просто игнорировать новые теги** (прямая совместимость)
- * У каждого типа в Protobuf есть значение по умолчанию – поэтому **новая схема может читать старые данные** (обратная совместимость)
- * **Удалять поля нельзя** (сломается прямая совместимость)
- * **Менять типы полей ограниченно можно**, но осторожно (например сделать `int32` → `int64`)

ЗАЧЕМ НУЖНА ПРЯМАЯ СОВМЕСТИМОСТЬ?



ПРЕДСТАВЬТЕ, ЧТО ВЫ ОБНОВИЛИ СХЕМУ

- * И раскатили апдейт – но это не моментальная операция
- * Очень вероятно, что некоторое время будут клиенты со старой схемой
- * А если вы раскатываете обновление частями – то вероятность становится 100%
- * Прямая совместимость позволяет старому коду работать с новыми данными

РЕЗЮМИРУЕМ ПО PROTOBUF

- * Данный формат представляет собой уже более сложное решение, нежели MessagePack и **просто так перейти на него с JSON уже не получится**
- * Взамен мы получаем еще более компактное представление **и комплексное решение проблем совместимости**
- * Использование схемы и статической компиляции хорошо сказывается **на производительности сериализации и десериализации**
- * Но теперь нужно **загружать сгенерированные библиотеки на клиент**
- * **Отладка и разработка стали сложнее**

ЧТО ОСТАЛОСЬ ЗА КАДРОМ

- * Кроме Protobuf в семействе бинарных форматов **очень много достойных кандидатов**, но время лекции ограничено
- * Например, [Apache Avro](#) (со схемой данных, очень хорош для выгрузки из БД)
- * Или [CBOR](#) (без схемы данных)
- * Оставляю вам это на самостоятельное изучение

ПОСЛЕДНЕЕ, ЧТО ХОЧЕТСЯ СКАЗАТЬ

- * Все разбираемые до этого форматы работали с **явной фазой сериализации и десериализации**
- * Но мы **можем избавиться от такой необходимости** если использовать подход с View над сырыми бинарными данными
- * Такой подход особенно актуален, **когда поток данных огромен**
- * Популярным решением с таким подходом является [Flatbuffers](#)
- * Или вы можете **реализовать своё решение под задачу**

FLATBUFFERS

- * **Концептуально похожи на Protobuf:** используется схема (fbs-файл, IDL), есть поддержка прямой и обратной совместимости
- * На основе схемы **генерируется файл на конечном ЯП**
- * Но вместо преобразования из буфера в объекты ЯП **реализует интерфейс View**
- * **Данные всегда остаются в форме буфера,** поэтому время сериализации и парсинга – константы
- * Это очень похоже на то, **что мы делали на открытом уроке**

КОГДА ОНО НУЖНО

- * Может показаться, что использование View подхода лучшее решение, но это далеко не так – есть лишь решения под задачу
- * С подходом на View **будут проблемы с изменениями данных переменного размера**
- * Или **с частой конвертацией данных при чтении/записи**
(например, если в буфере хранятся строки в отличной кодировке)
- * Выбор формата должен быть обдуманным и взвешенным решением

second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

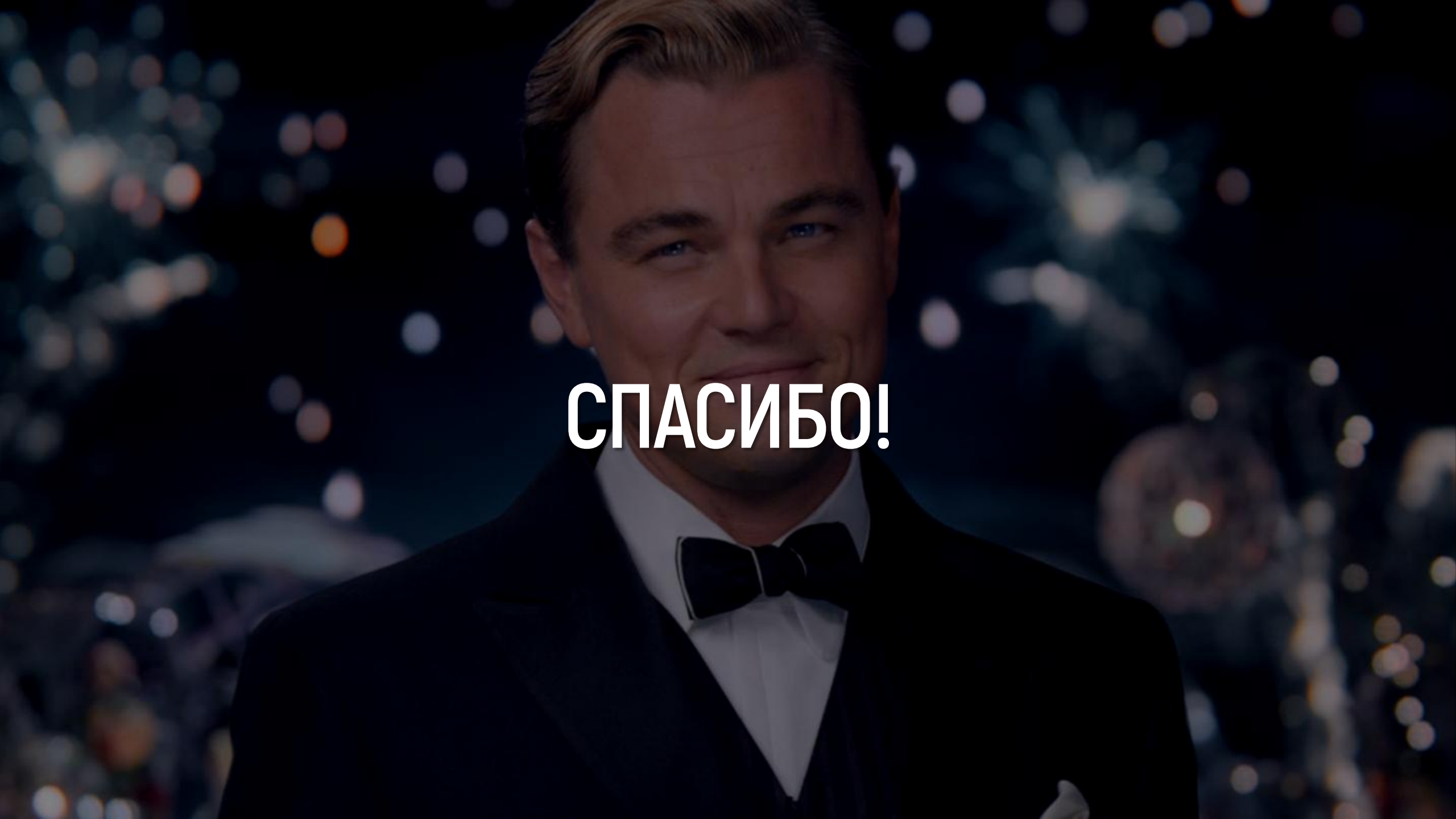
СТОЯНА, СТОЯНААА!

ВЫДЫХАЕМ

- * Лекция вышла насыщенной и **затронула много важных аспектов**
- * **Не спешите проглотить эту информацию** – дайте время осесть ей в голове
- * Лучшие там, где сложно

ПОДВЕДЕМ ИТОГИ

- * Когда у нас возникает **потребность сохранить или передать файл**, то мы сталкиваемся с понятием форматов данных
- * Можно условно **разделить все форматы на текстовые и бинарные**
- * И на **форматы для «обычных» данных или «специальных»** (конфики, разметка)
- * Реализация формата **полагаться на наличие схемы, так и её отсутствие**
- * И предполагать **явные фазы сериализации/десериализации** или работать как View
- * Мир больше, чем JSON 😊

A close-up portrait of Leonardo DiCaprio, smiling slightly, wearing a dark tuxedo with a white shirt and a dark bow tie. The background is dark with out-of-focus bokeh lights in various colors (white, yellow, orange, blue), suggesting a night-time event or fireworks.

СПАСИБО!